

De LFI a ejecucion (RCE)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Es el salto de 'puedo leer' a 'puedo ejecutar'. Una lfi por si sola solo lee, pero si consigues que el fichero incluido contenga código PHP que tu has metido antes, ese código se ejecuta con los permisos del servidor web y obtienes ejecución remota de comandos.

include no solo lee: interpreta PHP. Si logras escribir `<?php system($_GET['c']); ?>` en algún sitio del servidor y luego apuntas la LFI a ese sitio, el include ejecuta tu payload. Los sitios clásicos donde puedes 'inyectar' ese contenido son ficheros que el servidor escribe con datos tuyos: logs de acceso, ficheros de sesión, cabeceras guardadas o subidas de fichero mal filtradas.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ya confirmaste lectura con LFI y además existe algún fichero cuyo contenido puedas influir: el User-Agent acaba en un access.log legible, hay ficheros de sesión en /var/lib/php, o hay un formulario de subida. Sin un punto de inyección controlable, la LFI se queda en lectura.

Cómo identificarla

- Puedes leer un log (access.log, auth.log) vía la LFI: el log refleja datos que tu envías, como el User-Agent.
- Existe una subida de ficheros que no valida bien la extensión o el contenido.
- Puedes fijar el valor de una cookie de sesión y localizar su fichero en disco.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Manda un User-Agent con PHP mínimo (`<?php system($_GET['c']); ?>`) y luego incluye el access.log con `?c=id`.

Evidencia esperada: La salida de id aparece incrustada entre las líneas del log. Eso confirma ejecución, no solo lectura.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -s '$URL/?page=../../../../../../var/log/apache2/access.log&c=id'
```

Objetivo y contexto: Log poisoning: antes mandas PHP en tu User-Agent (queda escrito en el access.log) y ahora incluyes ese log vía la LFI. El servidor interpreta tu PHP y ejecuta el comando de ?c=.

Resultado esperado: La salida de id incrustada entre las líneas del log confirma ejecución. Fíjate en el usuario (www-data): es con quien tendrás la shell al escalar a reverse shell.

Anatomía de los payloads

La primera prueba es `curl -s '$URL/?page=../../../../../../var/log/apache2/access.log&c=id'`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

La salida de id aparece incrustada entre las líneas del log. Eso confirma ejecución, no solo lectura.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Elige el vector de inyección: log poisoning (mandas PHP en el User-Agent y lo ejecutas incluyendo el log), sesión, o subida.
- Inyecta un payload mínimo que ejecute un comando parametrizado, y confirma con id o whoami que se ejecuta.
- Convierte la ejecución de un comando en una shell interactiva: lanza una reverse shell (reverse-vs-bind) y estabilízala (tty-stabilization).
- Con shell ya dentro, cambia de fase: orientate y busca escalada (privesc-modelo). Los comandos concretos están en la sección de práctica (ver comandos).

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No hay ningún fichero en el servidor cuyo contenido controles (logs no legibles, sin uploads, sin sesiones en disco).

- El servidor no interpreta PHP en el fichero incluido (p.ej. solo lo muestra como texto).
- open_basedir u otra restriccion impide llegar a la ruta del fichero envenenado.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Leer ficheros no es el final: si logras que el servidor incluya contenido que tu controlas, ejecutas código. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.

2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- Concepto fuente de THM Fieldbook (lfi-a-rce)
- Referencia técnica externa

Navegación

Anterior: Construcción de payloads. Índice: curso. Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.